Zachary Jock

MTH 437: Introduction to Numerical Analysis I

October 29, 2025

[95]:
```
import numpy as np
import math
import matplotlib.pyplot as plt
import pandas
import scipy
import time
```

[96]:
```
data = np.loadtxt("A.dat", delimiter=",")
```

[97]:
```
data.shape
```

[97]: (1600, 1600)

0.1 Steepest Descent Methods for Solving a Linear System

Suppose we had an A matrix that is known to be symmetric positive definite (i.e. $A^T = A$ and $x^T Ax > 0 \ \ \forall x \neq 0$), and this A is apart of a linear system $Ax = b$. If we want to solve for x, we can redefine the system of equations as a function of x such that

$$f(x) = \frac{1}{2}x^T Ax - b^T x$$

To approximate the solution vector x, we want to take the negative gradient where the function will converge closest to 0 (or rather our defined ϵ tolerance).

$$\nabla f = Ax - b \ \rightarrow \ -\nabla f = b - Ax$$

We can call this negative gradient d to represent the direction (Or more appropriately we would refer to this negative gradient as the "residual" in context with algorithms we will define later).

$$d_0 = r_0 = b - Ax_0$$

This direction or residual vector would be substituted back in for the solution vector x and iterated upon. α_n represents an optimal step size that determines the distance for the residual to move in.

$$x_{n+1} = x_n + \alpha_n d_n$$

$$\alpha_n = \frac{d_n d_n}{d_n(Ad_n)}$$

$$d_{n+1} = d_n - \alpha_n A d_n$$

In an algorithm, we would keep iterating the step size α_n, the residual d_n and the approximation x_n until the relative error $\frac{||d_n||}{b}$ converges to the designated ϵ_{tol}.

[98]:
```
def descent(A,b,x,tol,iteration):
    d = b - A @ x #descent direction (residual) found with dot product
    bn = np.linalg.norm(b) or 1.0 #length of b; if direction is zero then
    #force ||b|| as 1.0 to mitigate error
    err = np.linalg.norm(d)/ bn #compare direction residual to b
    k = 0 #iteration count
    i_l = list()
    err_l = list()
    for i in range(iteration):
        if err < tol: #If normal vectors of descent direction divided
            #by b is less than tolerance, stop the algorithm
            break
        #if the normal vector ratio is not less than tolerance,
        #we will keep updating the step size, direction and
        #x until the if-condition is satisfied
        a = (d @ d) / (d @ (A @ d)) # define step size scalar
        x = x + (a * d) #iterate solution vector x with direction times
        #the step scalar
        d = d - a * (A@d) #update direction with new step size
        err = np.linalg.norm(d)/ bn #update residual/direction error
        k += 1
        i_l.append(k) #log iteration count
        err_l.append(err) #log error per iteration
    plt.plot(i_l,err_l)
    plt.title("Steepest Descent Algorithm")
    plt.xlabel("# of Iterations")
    plt.ylabel("Residual Error")
    plt.show()
    return i_l[-1],err_l[-1]
```

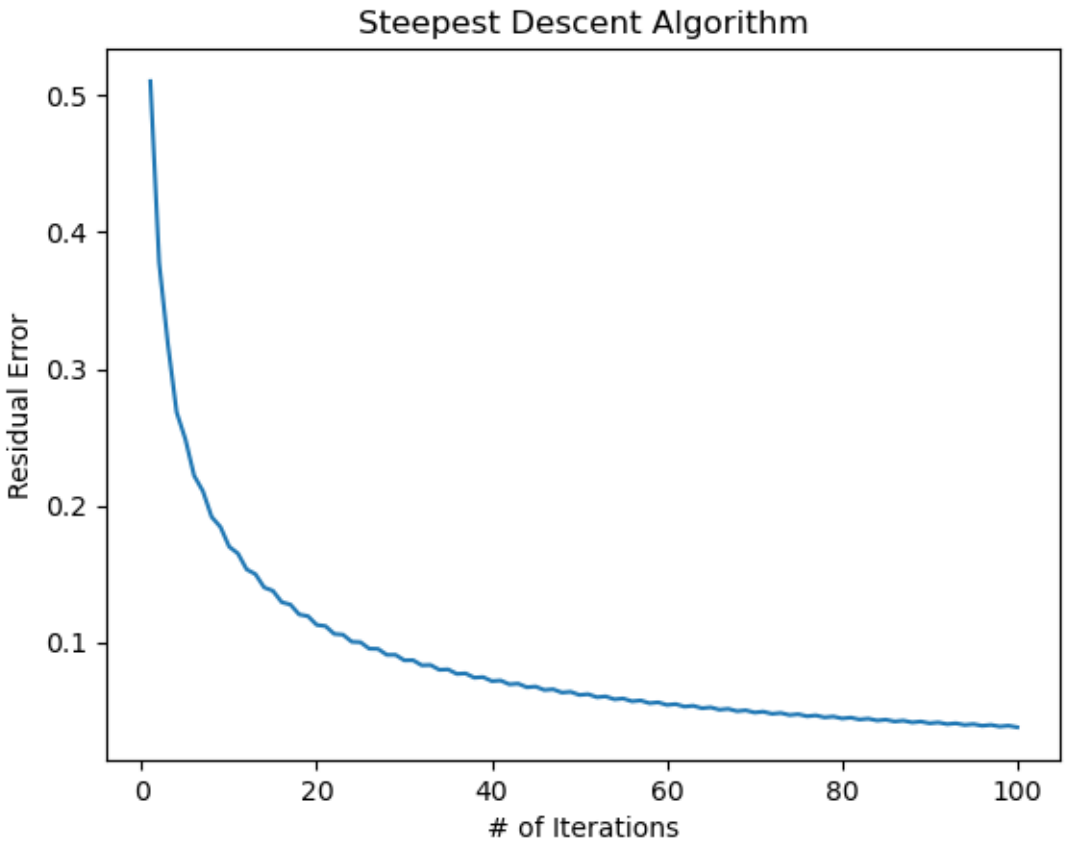
To set up our linear system, we will define an x vector of ones, and an x_0 vector of zeroes. We will use x to establish an initial value for $Ax = b$, and we would use x_0 as our initial guess for the descent algorithm (intuitively, the first attempt for a linear system would be to find the trivial solution $Ax = 0$).

[99]:
```
x = np.ones(1600)
```

[100]:
```
x_0 = np.zeros(1600)
```

[101]:
```
b = data@x
```

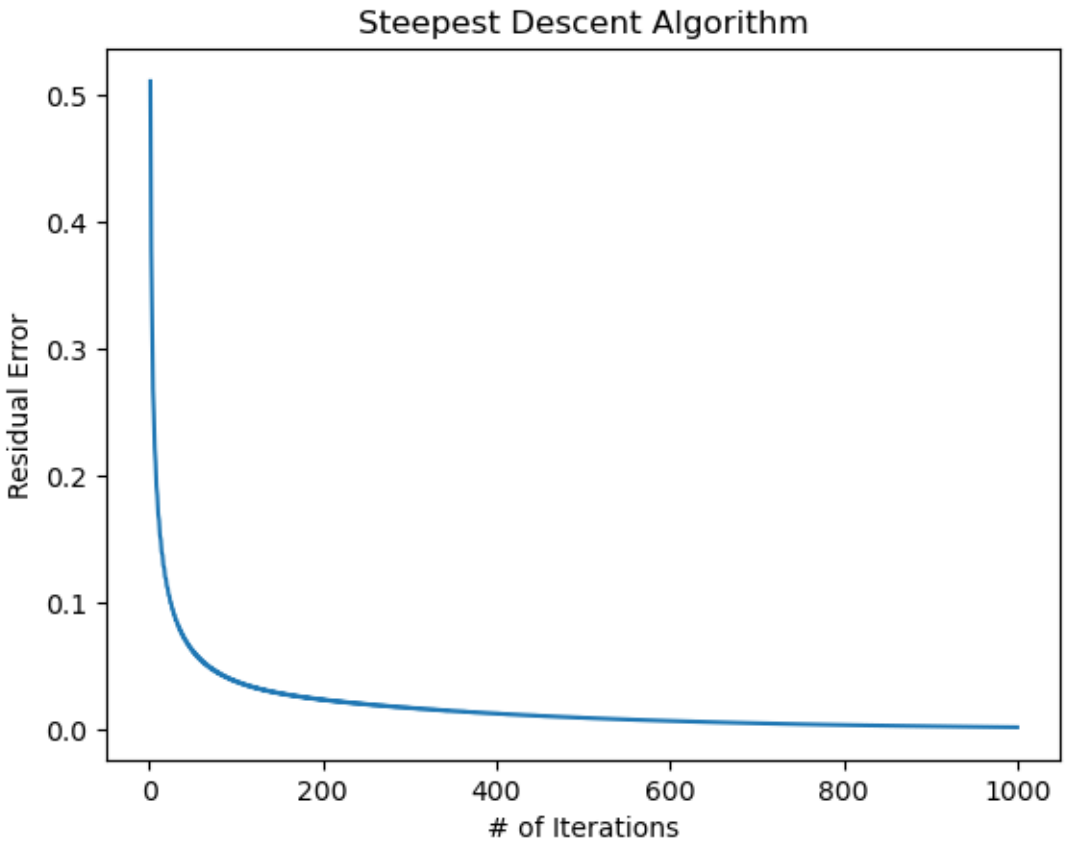
[102]:
```
descent(data,b,x_0,1e-10,100)
```



[102]:
```
(100, np.float64(0.037733654750743945))
```

A problem that we encounter with the steepest descent algorithm is that residual error is relatively high to the number of iterations we have defined in the parameters. Furthermore, the algorithm runs through every iteration defined in the parameters, which persists if we increase the iteration number to 1,000.

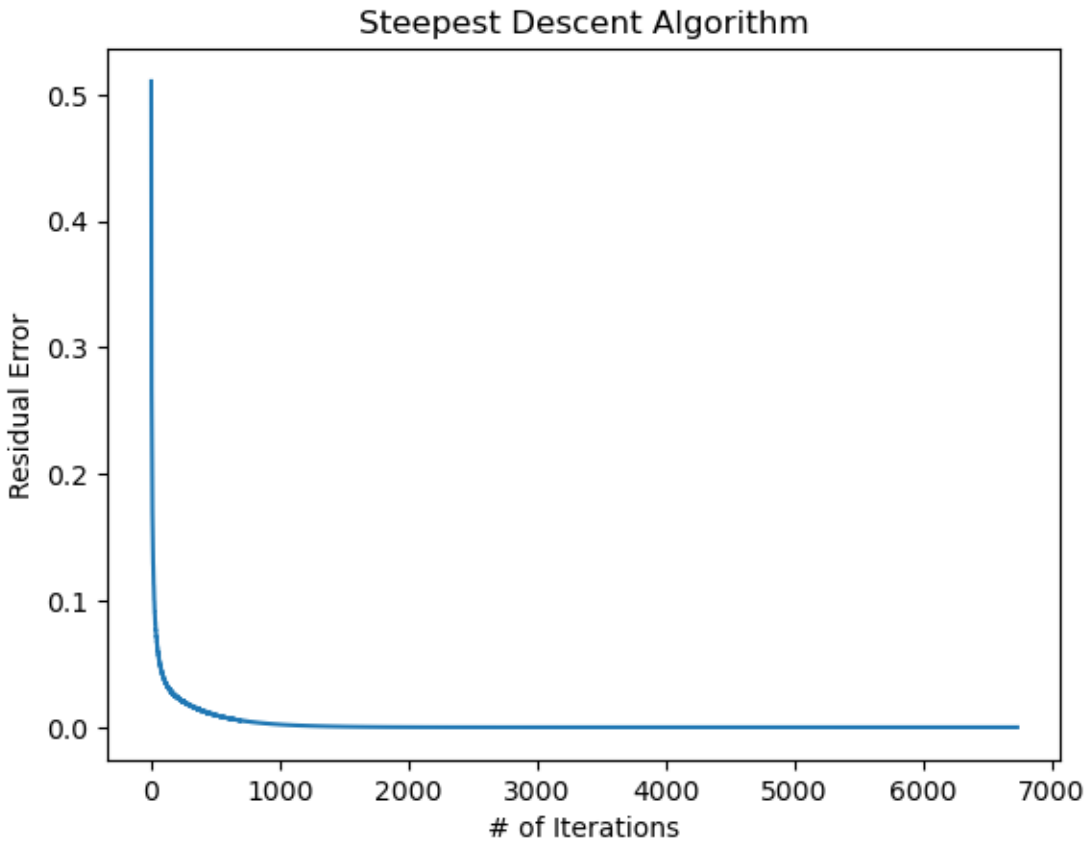
[103]:
```
descent(data,b,x_0,1e-10,1000)
```



[103]:
```
(1000, np.float64(0.002171085963815141))
```

If we increase the order of magnitude once more, we are able to obtain a residual error rate that is less than the ϵ_{tol}, but only after 6,732 iterations.

[104]:
```
descent(data,b,x_0,1e-10,10000)
```



[104]:
```
(6732, np.float64(9.966376063577965e-11))
```

For our matrix A we are able to obtain results within seconds, but out of principle we would want to refine our algorithm so that we could obtain the same or less number for the residual error, but with less iterations.

0.1.1 Jacobi-Preconditioned Conjugate Gradient Method

If we were to check the conditioning of our matrix A - that is, the factor at which the error rate is magnified for solving $Ax = b$ using NumPy, we will find that the number is quite high:

[105]:
```
np.linalg.cond(data)
```

[105]:
```
np.float64(680.6170700217114)
```

If we have a $cond(A) \approx 10^k$, we would expect to lose k numbers of accuracy for each iteration of x. In our case where $cond(A) \approx 10^3$, we should expect to lose 3 digits of accuracy per iteration. This is quite small in comparison to our machine-ϵ of 10^{-15}, but this would explain how we had to raise our iteration count to over 6,000 in our to find an error rate that superceded our defined ϵ_{tol}. (Sauer)

A possible solution would be to introduce a preconditioning multipler M into the linear system such that

$$M^{-1}Ax = M^{-1}b$$

For this example we would want to use the Jacobi preconditioner, where $M = D$, where D is the diagonal of A.

If we compute $cond(D^{-1}A)$, we significantly lower condition number of 1.0. Based on our scale of $cond(A) \approx 10^k$, we would expect to lose nearly zero digits of accuracy for each iteration of x_n

[106]:
```
np.linalg.cond(np.linalg.inv(np.diag(np.diag(data)))*data) #perform np.diag␣
↪twice to get 2-D diagonal matrix
```

[106]:
```
np.float64(1.0)
```

This would address the problem with accuracy across iterations, but how would we address the efficiency of the steepest descent algorithm? In our first algorithm, our residual was the same as our search direction d_n, and would reset with each iteration. With the conjugate gradient method, we introduce a second residual z that is the first residual r with the preconditioner applied.

$$d_0 = z_0 = M^{-1}r_n$$

$$\alpha_n = \frac{r_n z_n}{d_n(Ad_n)}$$

$$r_{n+1} = r_n - \alpha_n A d_n$$

We will also want to define the search direction d_n and residuals separately, since for the conjugate gradient method we would want the current residuals and all previous ones to be orthogonal to each other. The orthogonal residuals allow us show that d_n is pairwise A-conjugate (i.e. $d_{n+1}^T A d_n$ is equal to 0).

To ensure that each direction d_{n+1} is pairwise A-conjugate, we can define a scalar β that ensures that the conjugacy is forced for all d_n. (Sauer)

$$\beta_n = \frac{r_{n+1}z_{n+1}}{r_n(z_n)}$$

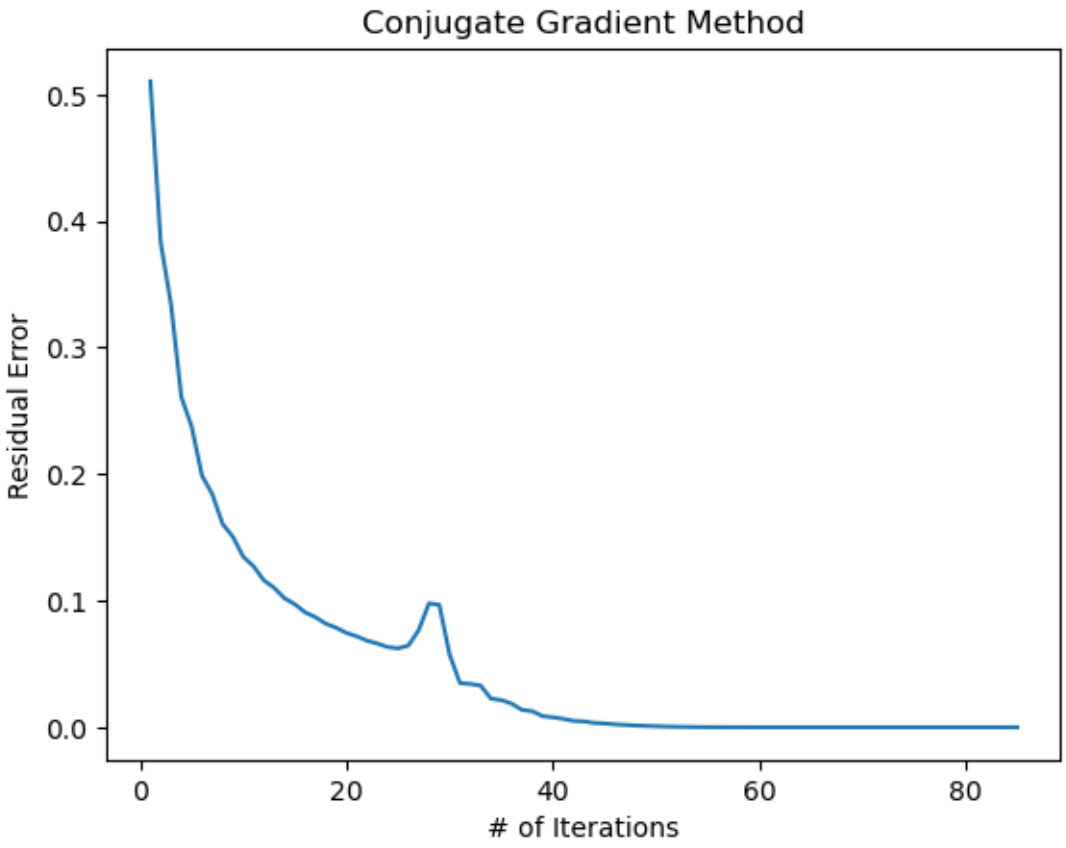
$$d_{n+1} = z_{n+1} + \beta_n d_n$$

Since we have the scalar β that stores the previous and current residuals and forces conjugacy for every d_{n+1}, the search direction is not repeated like it was for our initial steepest descent algorithm.

[107]:
```
def descent_cg(A,b,x,tol,iteration):
    r = b - A @ x #residual/negative grad.
    d = (1.0/np.diag(A))*r # jacobi preconditioned search direciton
    z = d #jacobi preconditioned residual
```

```
    bn = np.linalg.norm(b) or 1.0
    err = np.linalg.norm(d)/ bn
    k = 0
    i_l = list()
    err_l = list()
    for i in range(iteration):
        if err < tol: #If normal vectors of descent direction
            #divided by b is less than tolerance, stop the algorithm
            break
        #if the normal vector ratio is not less than tolerance,
        #we will keep updating the step size, direction and x until the␣
↪if-condition is satisfied
        a = (r @ z) / (d @ (A @ d)) #step size scalar
        x = x + (a * d) #iterate solution vector x with direction times
        #the step scalar
        r_new = r - a*(A@d) #iterate the unconditioned residual
        z_new = (1.0/np.diag(A))*r_new #iterate the preconditioned residual.
        beta = (r_new @ z_new)/(r@z) #conjugate scalar that combines r and z
        #residuals
        d = z_new + beta*d #update conjugate search direction
        #update residuals
        r = r_new
        z = z_new
        err = np.linalg.norm(r_new)/ bn
        k += 1
        i_l.append(k)
        err_l.append(err)
    plt.plot(i_l,err_l)
    plt.title("Conjugate Gradient Method")
    plt.xlabel("# of Iterations")
    plt.ylabel("Residual Error")
    plt.show()
    return i_l[-1],err_l[-1]
```

[108]:
```
descent_cg(data,b,x_0,1e-10,400)
```



[108]: `(85, np.float64(5.90203616288177e-11))`

Evidentally, we were able to solve the same linear system as before with just 85 iterations, which is a stark contrast from the 6,732 iterations we required for the steepest descent algorithm. Additionally, we were able to maintain the same residual error of $\epsilon = 10^{-11}$.

0.1.2 Pre-Conditioning with SSOR

Another method of preconditioning that we can use on our symmetric positive definite matrix is the Symmetric Successive Overrelaxation (SSOR) method. For this method, we first factor the matrix A into its diagonal, lower triangular and upper triangular such that $A = D + L + U$. From this we can define a multiplier M with an ω scalar. This ω parameter is known as the relaxation parameter, and can change the rate of convergence for the solution. (Sauer)

$$M = (D + \omega L)D^1(D + \omega U)$$

Solving for M^{-1} directly would generally be too resource-intensive, but we can do a backwards substitution using the factorized parts of A twice to obtain our preconditioned residual z.

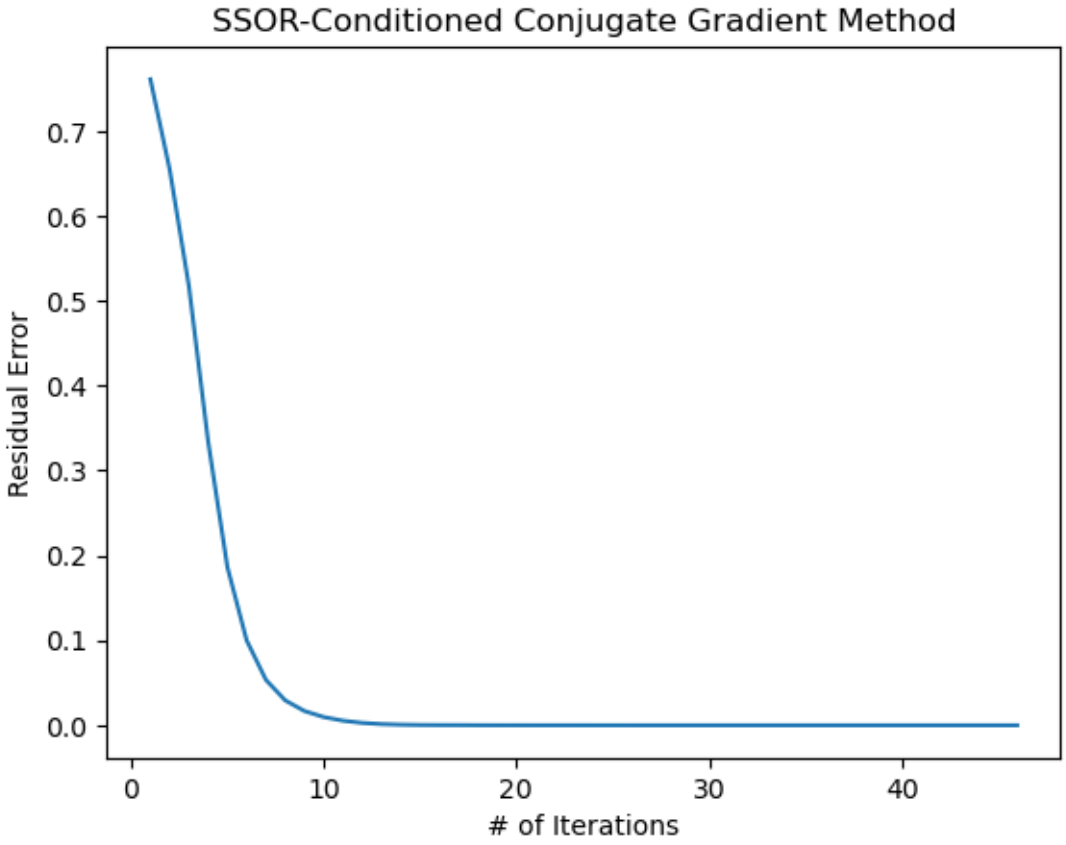
$$(I + \omega LD^{-1})c = v$$

$$(D + \omega U)z = c$$

$$z = M^{-1}v$$

[109]:
```
def descent_cg_sor(A,b,w,x,tol,iteration):
    r = b - A @ x #descent direction found with dot product
    #
    D = np.diag(np.diag(A))
    L = np.tril(A,-1) #lower triangular of A
    U = np.triu(A,1) #upper triangular of A
    #two-part back substitution to find M preconditioner
    #and residual z
    y = scipy.linalg.solve_triangular((D+w*L),r,lower=True)
    y = (1.0/np.diag(D))*y
    z = scipy.linalg.solve_triangular((D+w*U),y,lower=False)
    # Now that we have z, we define initial search direction d
    d = z.copy()
    bn = np.linalg.norm(b) or 1.0
    err = np.linalg.norm(d)/ bn
    k = 0
    i_l = list()
    err_l = list()
    for i in range(iteration):
        if err < tol: #If normal vectors of descent direction
            #divided by b is less than tolerance, stop the algorithm
            break
        #if the normal vector ratio is not less than tolerance,
        #we will keep updating the step size, direction and x until the␣
↪if-condition is satisfied
        a = (r @ z) / (d @ (A @ d)) #step size scalar
        x = x + (a * d) #iterate solution vector x with direction times
        #the step scalar
        r_new = r - a*(A@d) #iterate the unconditioned residual
        #run two-part substitution with new residual to update z
        y = scipy.linalg.solve_triangular((D+w*L),r_new,lower=True)
        y = (1.0/np.diag(D))*y
        z_new = scipy.linalg.solve_triangular((D+w*U),y,lower=False)
        beta = (r_new @ z_new)/(r@z) #conjugate scalar
        d = z_new + beta*d #update search direction
        #update residuals
        r = r_new
        z = z_new
        err = np.linalg.norm(r_new)/ bn
        k += 1
        i_l.append(k)
        err_l.append(err)
    return i_l, err_l #we'll do plotting and result output later
```

[110]:
```
i,y = descent_cg_sor(data,b,2.0,x_0,1e-10,400);
plt.plot(i,y)
plt.title("SSOR-Conditioned Conjugate Gradient Method")
plt.xlabel("# of Iterations")
plt.ylabel("Residual Error")
plt.show()
print(f'iterations: {i[-1]}',f'Error: {y[-1]}')
```



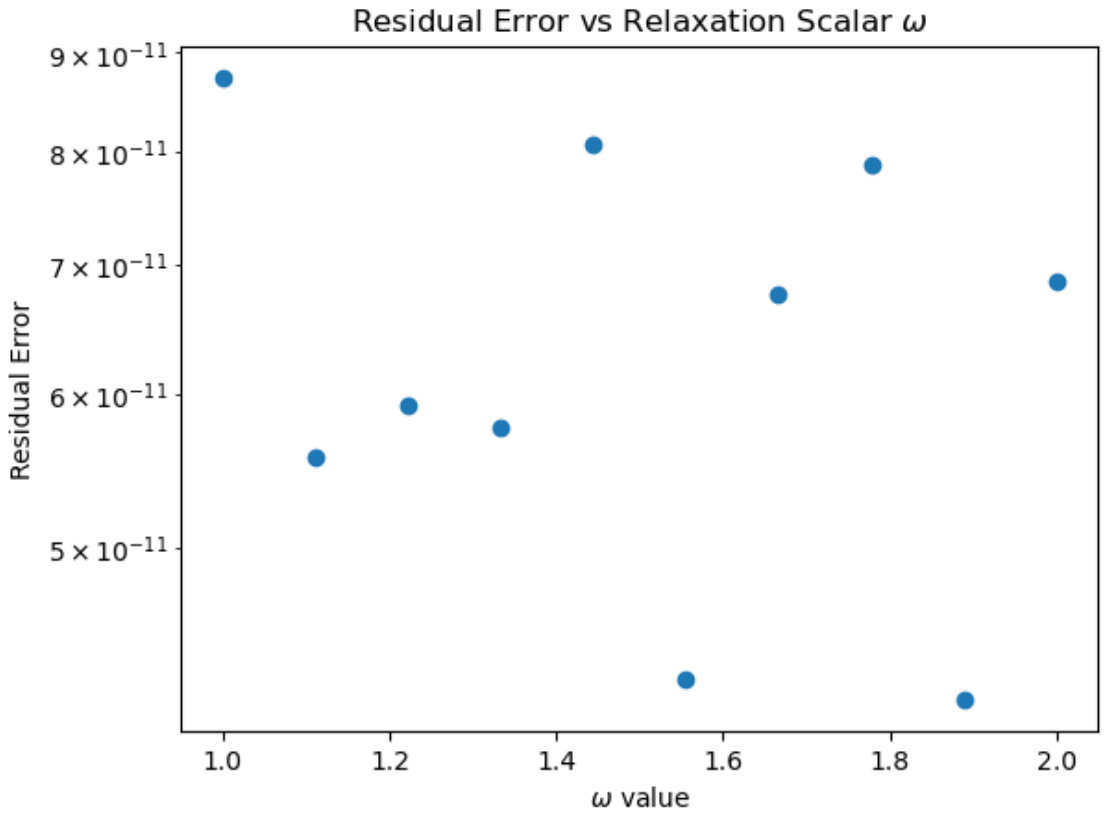

```
iterations: 46 Error: 6.869028182553364e-11
```

With our SSOR precondition we once again were able to maintain a nice residual error of $\epsilon = 10^{-11}$, but we have also halved the number of iterations needed to solve the linear system.

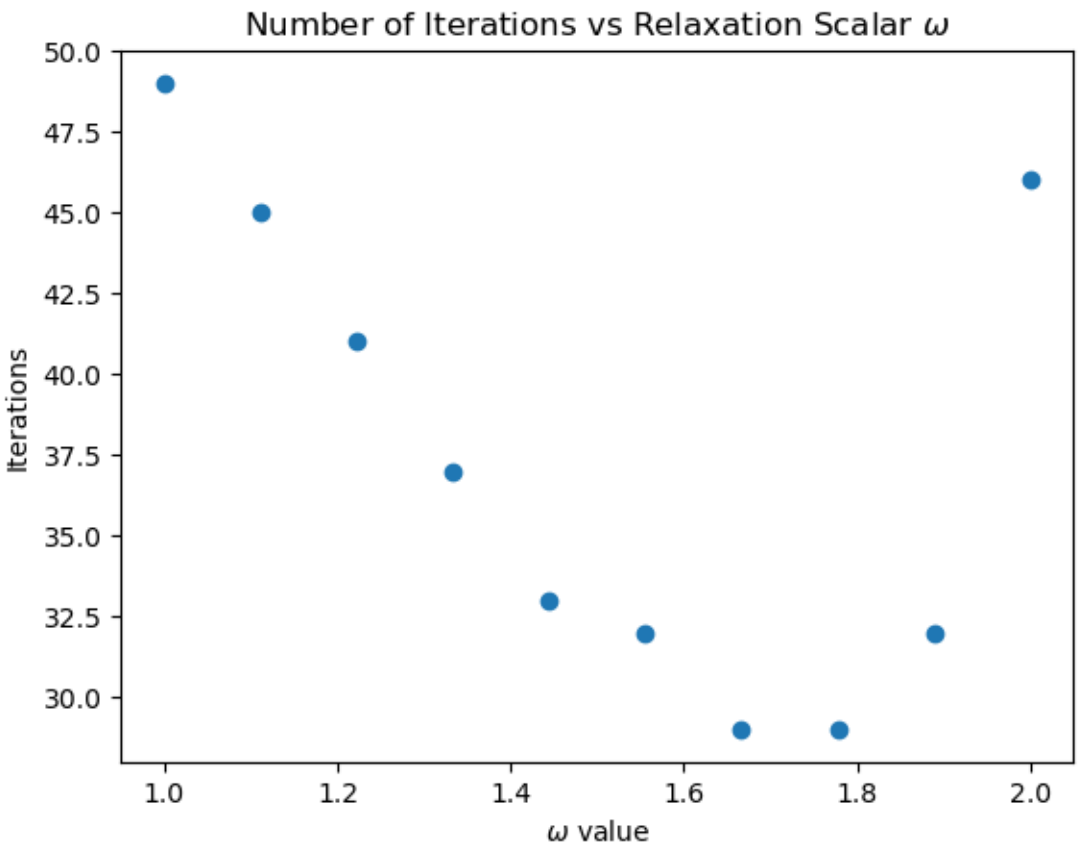
We had previously mentioned that SSOR has a special relaxation scalar ω that changes the rate of convergence for the solution x_n. As an additional thought excerise we can benchmark particular values of ω and how they change the error, the number of iterations and the function runtime,

[111]:
```
wx = np.linspace(1.0,2.0,10)
erl = [] #residual error for each run of descent_cg_sor()
for w in wx:
    i,y = descent_cg_sor(data,b,w,x_0,1e-10,100);
```

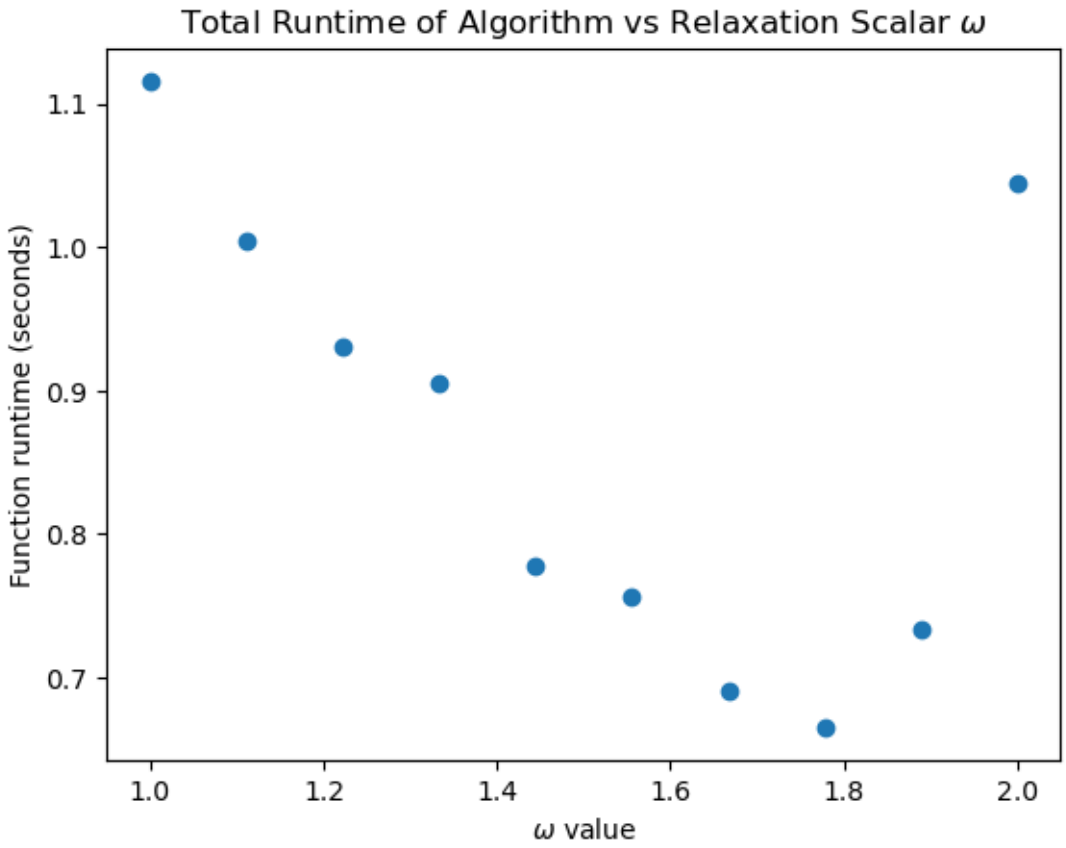
```
    erl.append(y[-1])
plt.scatter(wx, erl)
plt.yscale('log')
plt.title("Residual Error vs Relaxation Scalar $\omega$")
plt.xlabel("$\omega$ value")
plt.ylabel("Residual Error")
plt.show()
```



[112]:
```
wx = []
ix = [] #record of total iterations per descent_cg_sor()
for i in np.linspace(1.0,2.0,10):
    il,y = descent_cg_sor(data,b,i,x_0,1e-10,100)
    wx.append(i)
    ix.append(il[-1])
plt.scatter(wx, ix)
plt.title("Number of Iterations vs Relaxation Scalar $\omega$")
plt.xlabel("$\omega$ value")
plt.ylabel("Iterations")
plt.show()
```

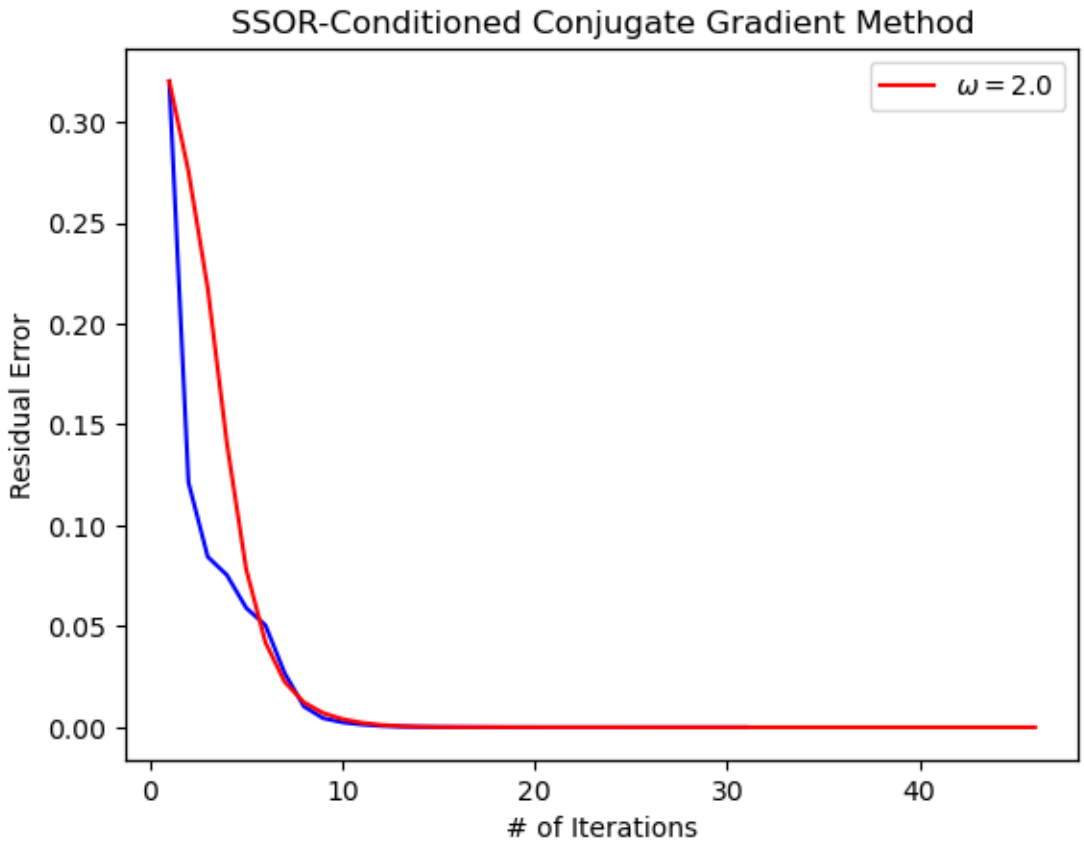


[113]:
```
tx = [] #record time for each run of descent_cg_sor()
wx = []
for i in np.linspace(1.0,2.0,10):
    start = time.time()
    _i_l,error = descent_cg_sor(data,b,i,x_0,1e-10,100)
    end = time.time()
    wx.append(i)
    tx.append(end-start)
plt.scatter(wx, tx)
plt.title("Total Runtime of Algorithm vs Relaxation Scalar $\omega$")
plt.xlabel("$\omega$ value")
plt.ylabel("Function runtime (seconds)")
plt.show()
```



Through these benchmarks, we can see that an ω value of 1.6 seems to have the fastest run time, lowest residual error and least number of iterations.

[114]:
```
fig, ax1 = plt.subplots() #defining plt with axes to display two functions␣
 ↪simultaneously
i,y = descent_cg_sor(data,b,1.6,x_0,1e-10,400)
i2,y2 = descent_cg_sor(data,b,2.0,x_0,1e-10,400)
ax1.plot(i,y, color='b')
ax2 = ax1.twinx()
ax2.plot(i2,y2,color='r',label="$\omega = 2.0$")
ax2.yaxis.set_visible(False)
ax1.set_xlabel("# of Iterations")
ax1.set_ylabel("Residual Error")
plt.title("SSOR-Conditioned Conjugate Gradient Method")
plt.legend()
plt.show()
print(f'iterations for omega = 1.6: {i[-1]}',f'Error for omega = 1.6: {y[-1]}')
print(f'iterations for omega = 2.0: {i2[-1]}',f'Error for omega = 2.0: {y2[-1]}')
```




```
iterations for omega = 1.6: 31 Error for omega = 1.6: 3.670873127692796e-11
iterations for omega = 2.0: 46 Error for omega = 2.0: 6.869028182553364e-11
```

Evidentally, using $\omega = 1.6$ in our function is faster than $\omega = 2.0$. Creating benchmarks using the parameters in our system helps refine our approximation if we have already sufficiently refined our algorithm.

0.1.3 Bibliography

- Sauer, T. (2012). Numerical analysis (2nd ed.).
- Wikipedia contributors. (2025a, September 21). Gradient descent. Wikipedia. https://en.wikipedia.org/wiki/Gradient_descent
- Wikipedia contributors. (2025b, October 25). Conjugate gradient method. Wikipedia. https://en.wikipedia.org/wiki/Conjugate_gradient_method

AI Disclosure: AI was used to assist with syntax and explanation of concepts from the Sauer textbook. All of the narrative and coding comments were written by myself.